# CMPT 383 Quiz 2 (Go)

# Summer 2026



| | |
|---|---|
| **Duration** | 30 minutes for the entire exam. |
| **Coding Questions** | For coding questions, use pseudocode in the style of the language the question is about. |
| **Aids allowed** | Pencil or pen, and eraser. No notes, no papers, no books, no computers, no calculators, no cheat sheets, etc. |
| **During the exam** | Raise your hand if you would like to speak with a proctor. To be fair to all students, we don't answer questions about the exam. Choose the answer based on your best understanding of the question. Re-reading the question may help. |

# Go Quiz

For each question, clearly write “true” or “false”. Correct answers are worth 1 mark. Incorrect answers, unanswered questions, or unreadable answers are worth 0 marks.

1. Go has **exceptions** that can be thrown and caught, similar to Python or C++.
   **False**: Go does not support exceptions. Errors are handled using multiple return values and panic/recover.
2. Go has a built-in **map** data type that is similar to Python’s dictionary data type.
   **True**: Go has an efficient and easy to use built-in map type.
3. Go has **pointers**.
   **True**: it has pointers, but no pointer arithmetic.
4. Go supports **closures**.
   **True**: So, for instance, returned functions can refer to variables outside the function.
5. Like Python, Go uses **garbage collection** to manage dynamic memory.
   **True:** No dangling pointers or memory leaks.
6. A Go **interface** consists of a set of method signatures and data fields.
   **False**: Go interfaces have methods only, and no data fields.
7. In Go, for a type to implement an interface, the type must **explicitly** indicate that it implements the interface (e.g. `PrinterType implements ReaderInterface`).
   **False**: Go types automatically implement the interface if they have the correct methods.
8. The standard way for concurrent go-routines to communicate to each other in a Go program is by using the built-in data type `pipe`.
   **False**: The correct data type for go-routine communication is channel.
9. Support for concurrent programming is one of Go’s strongest features, and it is not uncommon for a Go program to manage 100s or even 1000s of go-routines.
   **True**: Go is a good language for writing things like web servers.
10. This is a correct function signature for a Go function that uses generics to test if the value x is in a slice called `data`:
    `func Contains[T any] (x T, data []T) bool`
    **False**: T should be *comparable* (i.e. work with ==), not type any.

11. Write a fragment of Go code that declares a new variable called `nums` initialized to be a slice (not an array!) of `int`s with the values 5, 8, 2, 1, and then use a **range-style for-loop** to print the contents of `nums` to the console. You can assume `fmt` has been imported and the code fragment is already in a proper package and function.

**Sample Solution**

```
	nums := []int{5, 8, 2, 1}
	for _, n := range nums {
		fmt.Println(n)
	}
```

12. Write a complete Go function that takes a slice of integers as input, and returns both the max number in the slice, and a Boolean flag that is `false` if the slice is empty (and the returned `int` can be anything), and `true` otherwise. It should work like this:

```
	m, ok := findMax(nums) // nums is a slice of ints already defined
	if ok {
		fmt.Println("Max:", m)
	} else {
		fmt.Println("No max found (slice is empty)")
	}
```

**Sample Solution**

```
func findMax(nums []int) (int, bool) {
	if len(nums) == 0 { return 0, false }
	m := nums[0]
	for _, n := range nums {
		if n > m { m = n }
	}
	return m, true
}
```